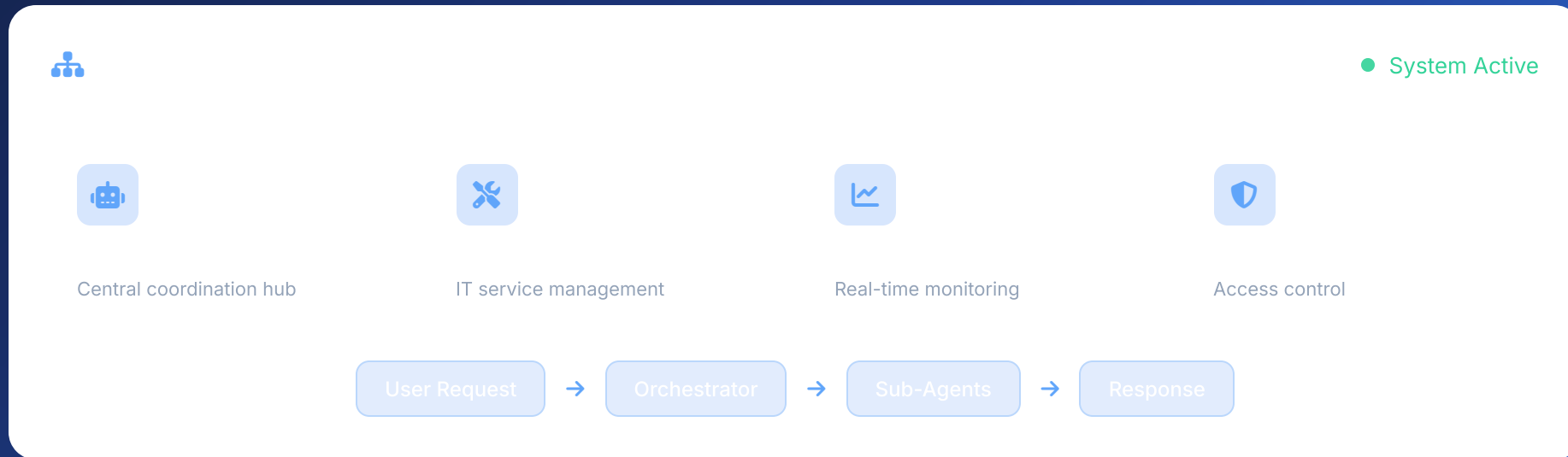


Enterprise IT Operations & Service Management Agentic Platform

AI-Powered Multi-Agent Architecture using Azure AI Foundry



AI-Powered

Intelligent agents with ML capabilities for automated decision making



Multi-Agent

Distributed architecture with specialized agent roles and communication



Azure Integration

Built on Azure AI Foundry for robust and scalable scalability and security



Enterprise AI Platform

Agent-to-Agent (A2A) Protocol

Open standard for interoperable agent communication across frameworks and stacks

What is A2A?

A2A is an open standard that defines how AI agents can discover each other, exchange information, and delegate tasks across different frameworks and platforms.

Discovery

Agent Cards via `/.well-known/agent-card.json`

Communication

Message-based, streaming via SSE

Tasks

Long-running processes supported

Security

OAuth2, Bearer/Basic auth

Key Features

-  **Cross-platform interoperability**
Works with different frameworks and languages
-  **Enterprise readiness**
Multi-tenancy, signed Agent Cards
-  **Web-aligned architecture**

Benefits



Reduced Integration

No custom glue code needed



Team Collaboration

Decoupled teams & stacks



Scalability

Easy to add/modify agents



Maintainability

Focused testing & debugging

Microsoft Integration

Agent Framework

Microsoft.Agents.AI.Hosting.A2A

A2A Client

A2AAgent/A2AClient

A2A Communication Flow

Microsoft Agent Framework A2A — Use Cases

Real-world applications of Agent-to-Agent protocol in enterprise scenarios



Cross-Team Compliance

Procurement agent hands off to partner compliance agent via A2A without stack rewrites. Ensures policy adherence across organizational boundaries.

Enterprise

Security

✓ Active



Customer Support Triage

Triage agent delegates to billing/network specialists using A2A endpoints. Intelligent routing based on query analysis.

Support

AI

🕒 24/7



Connected Analytics

Orchestrator queries research, retrieval, summarizer agents across vendors. Aggregates results for comprehensive insights.

Analytics

RAG

✓ Active



Legacy Integration

Wrap legacy service as an A2A agent. Partners discover via Agent Card and call securely without API changes.

Integration

Legacy

↻ Sync



Multi-Cloud Interop

Expose your agent; consume third-party A2A agents in the same workflow. Seamless cross-cloud communication.

Cloud

Hybrid

✓ Active



Workflow Automation

Automate complex business processes with A2A agents handling different stages of the workflow.

Automation

Workflow

🤖 Auto

</> Implementation Flow



Discovery



Connection



Communication



Validation

A2ACardResolver → GetAIAgentAsync() → RunAsync()

Multi-Agent Orchestration

Building orchestrator agent with planner + executor + validator sub-agents

Agent Roles & Responsibilities



Planner

Decomposes goals into steps; selects agents/tools



Executor

Performs tasks, runs tools, executes actions



Validator

Checks outputs vs acceptance criteria

Workflow Diagram



Implementation Guidance

Sequential

Use for step-by-step processing with clear dependencies

Concurrent

Use for parallel processing and independent analysis

Group Chat

Use for collaborative problem-solving

Code Example

C#

```
var workflow = new AgentWorkflowBuilder ()
    . WithPlanner (plannerAgent)
    . WithExecutor (executorAgent)
    . WithValidator (validatorAgent)
    . Build ();
```

Security Foundations

RBAC, Identity Management, Secure APIs, and Authentication/Authorization

RBAC & Identity Management



Least-Privilege Roles

Define granular permissions for agents, tools, and data access



Tenant Scoping

Isolate agent operations within tenant boundaries



Service Identity

Managed identities and service principals for agents



User Propagation

End-user identity propagation (on-behalf-of)

Authentication & Authorization

Implementation Guide

Fine-grained Scopes

Define specific permissions for tools and data access

Policy Enforcement

Implement policy enforcement points

```
// Secure API configuration
const config = {
  auth : {
    type : 'OAuth2' ,
    scopes : [ 'read:agents' , 'write:tools' ]
  }
};
```



Secure API Design



OAuth2/OIDC



mTLS



Monitoring & Audit



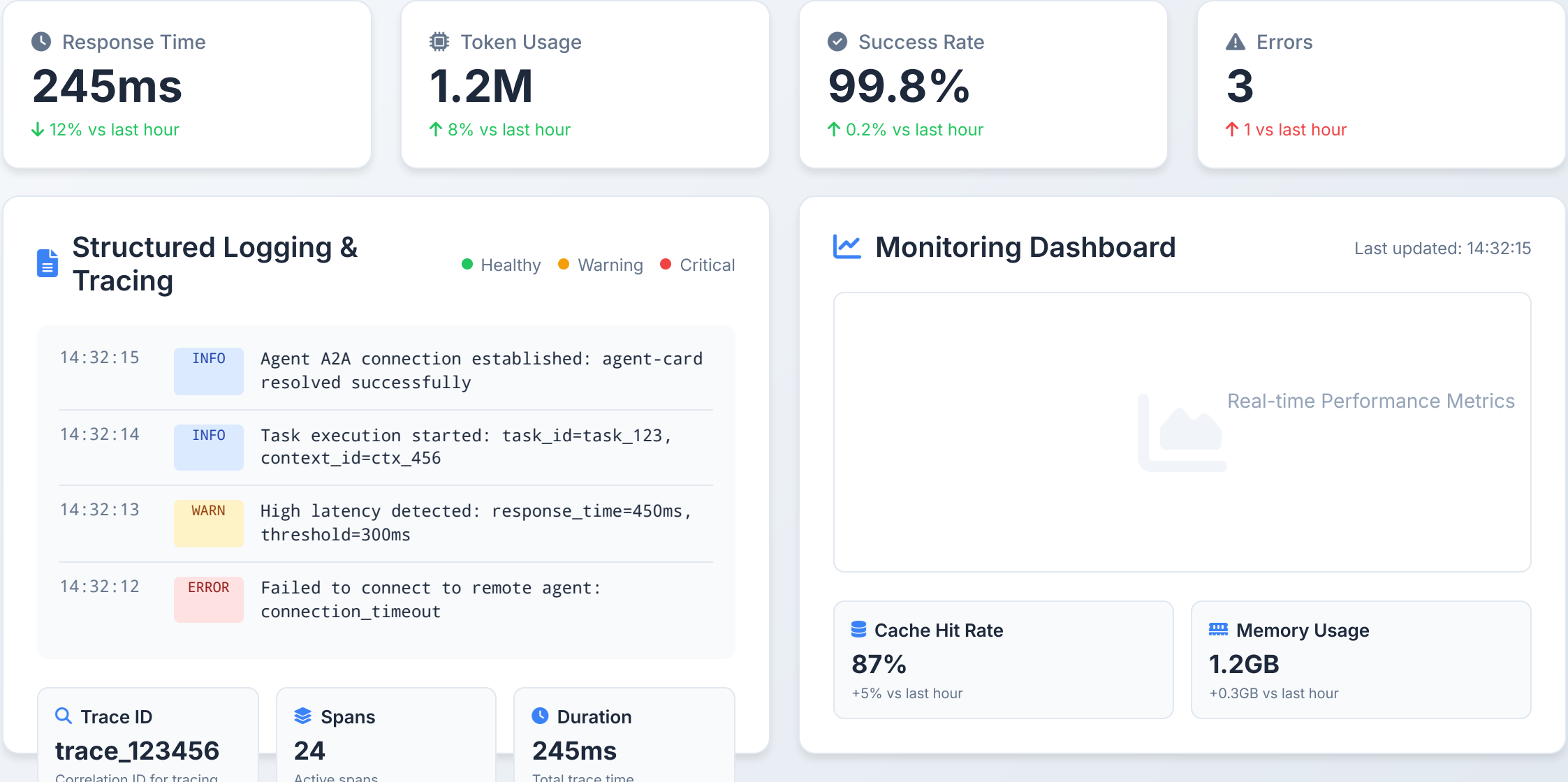
Audit Logs



Key Rotation

Observability & Monitoring

Structured logging, distributed tracing, and real-time monitoring for AI agents



🧠 Performance Optimization & Scaling

Latency optimization, caching strategies, parallel execution, and cost-performance tuning

📈 Performance Metrics

✓ Optimized

🕒 Response Time

245ms

↓ 12% vs last hour

🧠 Token Usage

1.2M

↓ 8% vs last hour

💾 Memory Usage

1.2GB

↓ 5% vs last hour

🗄️ Cache Hit Rate

87%

↑ 3% vs last hour

⚡ Optimization Strategies

🔗 Prompt Compression
Reduce token usage by 30%

Active

📦 Model Tiering
Right-size models per task

Active

🔄 Connection Pooling
Reduce connection overhead

Active

🗄️ Caching Strategies



Deterministic Keys
Hash-based caching



Semantic Caching
Similarity matching



RAG Caching
Chunk caching

Cache Check



Hit/Miss



Update

🔗 Parallel Execution

🔗 Fan-out Pattern
Distribute tasks across agents

Active

📋 Task Parallelism
Execute independent tasks

Active

🕒 Concurrency Control
Manage resource usage

Active

Cost & Scale Management — Enterprise AI Optimization

Cost-performance tuning, resource optimization, and scaling strategies for production AI systems



Cost-Performance Tuning

- ✓ Model tiering: Use smaller models for classification tasks
- ✓ Prompt compression: Reduce token usage by 40%
- ✓ Reasoning token budgets: Cap at 2K tokens

Cost savings: **\$12,450/month**

FinOps

Optimization



Resource Optimization

- ✓ Batching: Process 100+ requests/batch
- ✓ Cache hit-rate target: 85%
- ✓ Adaptive routing: Select optimal models



Caching

Batching



Scaling Strategies

- ✓ Autoscale: 10-100 instances
- ✓ Queue-based backpressure
- ✓ Multi-region failover

Current load:

78%

Auto-scale

Multi-region



FinOps Practices

- ✓ Per-tenant budget tracking
- ✓ Cost anomaly detection
- ✓ Real-time cost dashboards

Budget utilization: **67%**

Budgeting

Monitoring



Performance Metrics

- ✓ Latency: p95 < 200ms
- ✓ Throughput: 1000 req/s
- ✓ Availability: 99.9%

Success rate:

99.7%

SLA

Monitoring



Optimization Outcomes

- ✓ 30% cost reduction
- ✓ 50% latency improvement
- ✓ 2x throughput increase

ROI: **340%**

ROI

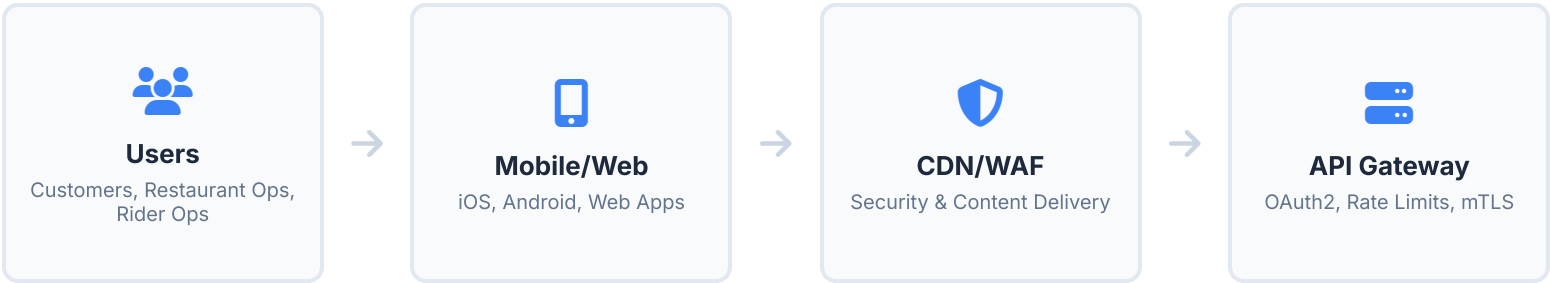
Success

Swiggy E2E Architecture

A2A Protocol & Microsoft Agent Framework Integration

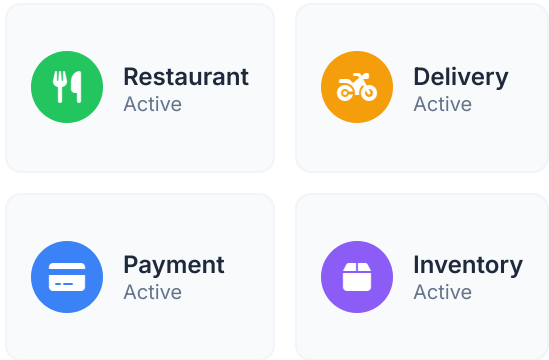
Production Ready

End-to-End System Flow

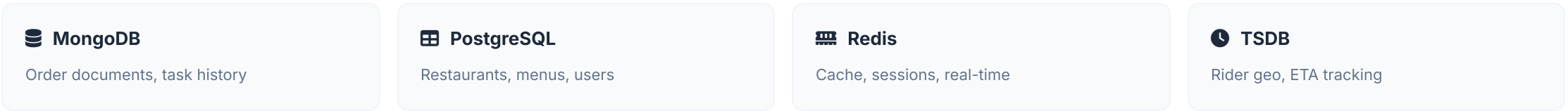


● System Status: **Operational**

Specialized Agents



Data Layer & Infrastructure



Monitoring & Observability



Swiggy Agent Ecosystem & A2A Communication

Order Management Orchestrator with Restaurant, Delivery, Payment, Inventory, and Customer Service Agents

Agent Ecosystem

● Active

Order Management Orchestrator

Central hub for coordinating all agent interactions

● Status: Active | Load: 78%



Restaurant Partner

Menu & order management



Delivery Fleet

Route optimization & tracking



Payment Agent

Transaction processing



Inventory Agent

Stock management



Customer Service

Support & queries



Security Agent

Auth & validation

A2A Communication Flow



1. Agent Discovery

Resolve Agent Cards (/.well-known/agent-card.json) for all agents



2. Task Delegation

Orchestrator sends tasks/send to appropriate agents with contextId



3. Streaming Responses

Real-time updates via SSE, status tracking, artifact generation



4. Validation & Completion

Final validation, artifact aggregation, task completion



Authentication & Security



OAuth2/Bearer



mTLS



Signed Cards

Swiggy Technical Implementation & Data Flow

Microsoft Agent Framework + A2A Protocol + MCP Server Integration

Client Request Flow

● Active



1. Mobile/Web Client

User initiates order request via iOS/Android/Web app



2. API Gateway

OAuth2/OIDC auth, rate limiting, input validation



3. Orchestrator Agent

Order Management Agent processes and routes request



Agent Framework Components



.NET A2ACardResolver

Agent discovery & resolution



A2AClient

Remote agent communication



Python agent-framework

A2AAgent/A2AExecutor



MCP Servers

Tool access & integrations



Database Architecture



MongoDB

Orders & Tasks



PostgreSQL

Restaurants & Menus



Redis

Cache & Session



TSDB

Time-series Data



Order Creation → Validation

Cache menu/pricing → Validate inventory



Observability & Security

245ms

LATENCY

99.9%

UPTIME

1.2M

TOKENS/MIN



OAuth2



mTLS



Tracing



Logging